Development: Intelligent Commit Message Generator

Aryaman Gautam
gautam8@illinois.edu
University of Illinois Urbana-Champaign
Champaign, Illinois, USA

Jonathon Eng Kiat Low
jelow2@illinois.edu
University of Illinois Urbana-Champaign
Champaign, Illinois, USA

Abstract

The "Smart Git Commit Message Pipeline" is a retrieval-augmented generation (RAG) system designed to automate the creation of high-quality, contextually relevant git commit messages. By utilizing a hybrid retrieval strategy,combining traditional keyword-based TF-IDF with semantic vector embeddings,the system identifies historical "diff-message" pairs from the commitpackft dataset that most closely resemble a developer's current changes. These candidates serve as few-shot examples for a LLM inference engine, which then synthesizes a precise commit message. Evaluation on a hold-out test set demonstrates that this hybrid approach significantly improves message relevance compared to zero-shot generation. The tool is implemented as a Google Colab notebook prototype and is intended for student developers, software engineering teams, and open-source contributors who want clearer commit histories with less manual effort.

1 Introduction

Effective version control relies on descriptive commit messages to maintain repository history and facilitate collaboration. However, developers frequently encounter "context-switching fatigue," where the cognitive load of summarizing complex code changes leads to cryptic or unhelpful messages (e.g., "fix bug" or "update"). This reduces the maintainability of software projects over time.

Our project addresses this by building an automated pipeline that "learns" from the existing history of high-quality commits. Unlike simple LLM-based generators that may hallucinate or ignore project-specific conventions, our tool uses a RAG architecture to provide the model with "ground truth" examples of how similar code changes have been described in the past. This ensures the output is not only grammatically correct but also contextually grounded in real-world development patterns.

This project is developed under the Development Track. The main artifact is a working software prototype implemented in a Google Colab notebook. The notebook supports dataset loading, diff preprocessing, retrieval index construction, top-k similar diff retrieval, prompt construction, LLM-based generation, and output inspection. The source code will be made available through a GitHub repository: https://github.com/gautam8-uofi/Smart-Git-Commit-Message-Pipeline

The rest of this report presents the SmartCommit system in detail. We first describe the system overview, intended users, major functions, inputs and outputs, and the role of retrieval in generation. We then explain the system architecture, implementation details, dataset preprocessing, retrieval methods, prompt construction, and structured LLM output. Finally, we provide usage instructions for the notebook prototype, evaluate the retrieval and generation components, discuss limitations and future work, and summarize each team member's contributions.

2 System Overview

The current implementation is built in a notebook format. This makes the prototype easy to run without requiring users to set up a complex local development environment. Users can execute the notebook cells to load the dataset, preprocess code diffs, construct retrieval indexes, test retrieval methods, and generate commit messages for held-out or user-provided diffs.

2.1 Intended Users

The primary users of SmartCommit are developers who use Git and want to improve the quality of their commit messages. This includes student developers, software engineering teams, and open-source contributors. These users may benefit from the tool when they want to quickly summarize code changes but still maintain readable and informative project history.

SmartCommit is especially useful in cases where developers are likely to write short or vague commit messages due to time pressure. By providing a suggested message, the system reduces the effort needed to write documentation for routine code changes. However, the tool is intended to assist rather than replace developers. The generated message should be reviewed and edited by the user before being used in an actual commit.

2.2 Major Functions

SmartCommit supports the following major functions:

(1) **Dataset loading and preparation.** The system loads a commit dataset containing old code, new code, and human-written commit messages. It then converts old/new code pairs into unified code diffs.
(2) **Diff preprocessing.** The system cleans the generated diffs by removing empty examples, filtering unusually long outliers, truncating long diffs, and standardizing the target commit message field.
(3) **Retrieval index construction.** SmartCommit builds retrieval indexes over the processed diff dataset. The current implementation supports both a TF-IDF retrieval baseline and an embedding-based retrieval method using Sentence-Transformers and FAISS (Facebook AI Similarity Search)
(4) **Top-k similar diff retrieval.** Given an input diff, the system retrieves the top-k most similar historical diffs from the retrieval set. Each retrieved example includes a similarity score, the retrieved diff, and its paired human-written commit message.
(5) **Prompt construction.** The retrieved diff-message pairs are inserted into a structured prompt template together with the current input diff. This gives the LLM concrete examples of how similar code changes were described.

(6) **Commit message generation.** The LLM generates a commit message for the input diff. The current output format includes the generated commit message, predicted commit type, confidence level, and a short reasoning summary.
(7) **Output inspection and evaluation support.** For analysis, the notebook displays the original input diff, the retrieved examples, the generated commit message, and the human reference message when available. This makes it easier to compare retrieval methods and evaluate the quality of the generated output.

2.3 System Inputs and Outputs

The main input to SmartCommit is a code diff. In the prototype, this diff can come from a held-out evaluation example in the dataset or from a user-provided diff. The retrieval database consists of processed historical diffs and their corresponding commit messages.

The main output is a suggested commit message. For debugging and evaluation, the system also outputs additional information such as retrieved examples, similarity scores, predicted commit type, model confidence, and a brief reasoning summary. This diagnostic output is useful because it helps users understand how the generated message was influenced by the retrieved examples.

2.4 Role of Retrieval in Generation

The retrieved examples are intended to improve generation in two ways. First, they provide the model with examples of how similar changes were summarized by humans. Second, they make the generation process more grounded by giving the model additional context beyond the current diff alone.

For example, if the input diff involves adding error handling, retrieved examples with commit messages such as “Handle missing configuration value” or “Fix exception handling in parser” can guide the model toward a more specific and appropriate message. Without retrieval, the model may still produce a reasonable message, but it may be more generic.

This retrieval-augmented design is especially relevant for an information retrieval system because the quality of the final generated message depends partly on the quality of the retrieved examples. Therefore, retrieval is not only an intermediate step but also a key contributor to the usefulness of the overall software tool.

3 System Architecture

The architecture follows a four-stage pipeline: Preprocessing, Hybrid Retrieval, Augmentation (Prompt Engineering), and Generative Inference. By decoupling the retrieval of historical context from the final generation, the system ensures that the output is grounded in real-world examples while maintaining the linguistic flexibility of an LLM.

3.1 Data Flow Overview

The pipeline begins when a user provides a raw git diff representing a set of staged changes. This diff is passed into the Retrieval Engine, which queries a vector database (semantic) and an inverted index (lexical) simultaneously. The top-k most relevant "diff-message" pairs are extracted and passed to the Prompt Orchestrator. Finally, the Inference Engine synthesizes these examples with the current diff to produce a final suggestion.

3.2 Hybrid Retrieval Engine

To ensure both technical precision and conceptual relevance, the system employs a "Dual-Path" retrieval strategy:

- Lexical Path (TF-IDF): This path focuses on keyword matching. It is highly effective for identifying changes in specific libraries, function names, or file paths. We utilize an N-gram approach ($n = 1, 2$) to capture common coding phrases.
- Semantic Path (Dense Embeddings): Using the all-MiniLM-L6-v2 model, the system encodes code diffs into a dimensional vector space of size 384. This allows the system to identify "intent" even when the variable names or syntax differ. Similarity is calculated using the cosine similarity metric:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

  This calculation determines the cosine of the angle between two vectors in the embedding space. A value near 1 indicates the vectors are nearly identical in direction (high semantic similarity), while a value near 0 indicates orthogonality (no similarity).

3.3 RAG Orchestration & Prompting

The "Augmentation" phase is the core innovation of the system. Instead of zero-shot generation, we use a Few-Shot In-Context Learning strategy. The orchestrator constructs a structured prompt that includes:

- System Persona: Defines the model as a professional software engineer
- Historical Context: k examples of similar diffs and their corresponding developer-written messages.
- Target Task: The current unseen diff.

This structure forces the model to perform "pattern matching" on the retrieved examples, ensuring the output adheres to the project’s typical verb tense, length, and technical depth.

3.4 Generative Inference Interface

For the generative backend, we utilize Llama 3 (8B) served via the Groq LPU (Language Processing Unit). This setup was chosen for two reasons:

- Inference Speed: Near-instant generation is critical for a developer tool meant to be used during the git commit workflow.
- Reasoning Capability: Despite its smaller parameter count compared to the 70B variant, Llama 3.1 8B shows high proficiency in understanding structured code changes when provided with sufficient context.

4 Implementation

This section details the technical realization of the pipeline, from initial data engineering to the integration of high-performance inference APIs.

4.1 Dataset and Preprocessing

SmartCommit uses historical code changes and their corresponding commit messages as the retrieval database for generation. For this project, we use the Python split of the CommitPackFT dataset. We chose the Python split to keep the project scope focused and consistent, while still using a realistic dataset of software changes.

Each data sample contains the old version of a code file, the new version of the code file, and the associated human-written commit information. Since SmartCommit operates on code diffs, we first convert each pair of old and new code contents into a unified diff. This diff becomes the document representation used for retrieval.

4.1.1 Diff Construction: The original dataset provides old and new file contents rather than a ready-made diff string. Therefore, we generate unified diffs using Python's difflib library. For each example, the old file contents and new file contents are split into lines and compared. The resulting unified diff captures added lines, deleted lines, and file-level metadata.

The simplified procedure is:

```
Input: old_code, new_code
Output: unified_diff
1. Split old_code into lines.
2. Split new_code into lines.
3. Use difflib.unified_diff to compare the two versions.
4. Join the resulting diff lines into a single diff string.
5. Store this string as the diff field for retrieval.
```

This step transforms each raw code-change example into a format that more closely resembles the input a developer would provide through a Git workflow.

4.1.2 Cleaning and Filtering: After generating diffs, we apply several cleaning steps to improve retrieval quality and reduce noise. We remove samples with empty diffs because they do not provide meaningful retrieval content. We also inspect the distribution of diff lengths and identify unusually long examples. Very long diffs can dominate retrieval scoring, increase prompt length, and make LLM generation less stable. To reduce this issue, we filter extreme outliers and truncate remaining diffs to a fixed maximum length.

We use the commit subject field as the target commit message because it is usually shorter and more suitable for generating concise commit titles than the full commit body. The final processed dataset keeps only the fields needed for this project: the generated diff and the corresponding commit message.

The preprocessing steps are:

```
1. Generate unified diff from old and new code.
2. Remove examples with empty diffs.
3. Filter extremely long diff outliers.
4. Truncate remaining diffs to a fixed maximum length.
5. Rename the commit subject field to commit_message.
6. Keep only the diff and commit_message columns.
```

4.1.3 Retrieval and Evaluation Split: To avoid evaluating the system on examples that are already indexed for retrieval, we split the processed dataset into two parts. A portion of the commitpackft dataset (10%) is reserved as the "test set." When evaluating, the system must retrieve "similar" examples from the remaining 90% and generate a message for the "unseen" test diff.

This split is important because it makes the evaluation more realistic. If the same diff appeared in both the query set and the retrieval database, the system could retrieve the exact same example and produce an overly optimistic result. By holding out evaluation diffs, we simulate the intended use case where a developer provides a new code change that the system has not seen before.

Table 1: Dataset preprocessing statistics

Statistic	Value
Original Python samples	56025
Empty diffs removed	52
Duplicate diffs observed	1309
Maximum diff length before filtering	4257
Mean diff length before filtering	781
Median diff length before filtering	569
Final processed dataset size	55705
Retrieval set size	50134
Held-out evaluation set size	5571

4.2 Lexical Retrieval Baseline (TF-IDF)

As a baseline for comparison, we implemented a lexical search using Term Frequency-Inverse Document Frequency (TF-IDF). This method treats code diffs as a "bag of tokens."

- Vectorization: We used TfidfVectorizer with a range of 1-2 n-grams to capture common programming sequences (e.g., if self, def __init__).
- Scoring: Given a new diff d_{new}, the similarity score S against an indexed diff d_i is calculated using the dot product of their L2-normalized TF-IDF vectors.

4.3 Semantic Retrieval and Vector Embeddings

To overcome the limitations of keyword matching, where two diffs might perform the same logic using different variable names, we implemented a semantic path.

- Embedding Model: We utilized the sentence-transformers/all-MiniLM-L6-v2 model. This model maps code snippets into a dense 384-dimensional vector space where functionally similar changes are spatially proximate.
- Indexing: The entire processed dataset was embedded and stored in a local tensor index. For any new query, we calculate the cosine similarity between the query embedding and the pre-computed index.

4.4 Comparison Between Retrieval Methods

The two retrieval methods have complementary strengths. TF-IDF is more transparent and often works well when code diffs share exact tokens. Embedding retrieval is less directly interpretable, but it may retrieve examples that are similar at a higher level of meaning.

In our system, both methods can be selected as the retriever for generation. This design allows us to compare whether retrieved examples from TF-IDF or FAISS lead to better commit messages. It also provides a useful baseline for evaluation: If the embedding-based method produces better retrieved examples or better generated

messages, this suggests that semantic retrieval is beneficial for this task.

Table 2: Comparison of retrieval methods

Method	Representation	Strength	Limitation
TF-IDF	Sparse token vector	Simple, fast, interpretable	Depends on exact token overlap
Embeddings + FAISS	Dense embedding vector	Captures semantic similarity	Less interpretable; requires embedding model

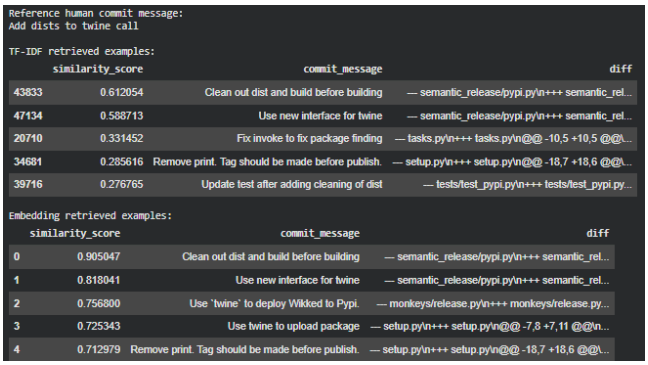

Figure 1: Example output of the two retrieval methods

4.5 Generative Integration and Prompt Engineering

4.5.1 Prompt Construction: The prompt is designed to guide the LLM toward producing commit messages that are concise, accurate, and useful. It contains four main parts:

(1) **Task instruction:** The model is told to generate a high-quality Git commit message from a code diff.
(2) **Writing guidelines:** The model is instructed to use imperative mood, keep the message concise, focus on the purpose of the change, and avoid unsupported details.
(3) **Retrieved examples:** The top-k retrieved diff-message pairs are included as examples of how similar code changes were described by human developers.
(4) **Current input diff:** The actual diff that requires a new commit message is placed after the examples.

The simplified prompt structure is:

```
You are an assistant that writes high-quality Git commit messages.

Guidelines:
- Use imperative mood.
- Keep the main message concise.
- Focus on the purpose of the change.
- Do not invent behavior unsupported by the diff.
- Use retrieved examples only as style/context references.

Retrieved similar examples:

Example 1:
Diff: ...
Human commit message: ...

Example 2:
Diff: ...
Human commit message: ...

Current diff:
...

Return structured JSON containing:
- commit_message
- commit_type
- confidence
- reasoning_summary
```

This prompt design allows the model to use similar historical changes as context without simply copying their commit messages.

4.5.2 Structured Output: To make the output easier to inspect and evaluate, SmartCommit asks the LLM to return a structured JSON response. The response contains:

Table 3: Structured output fields

Field	Description
`commit_message`	The generated commit message suggestion
`commit_type`	A predicted category such as `fix`, `feature`, `refactor`, `docs`, `test`, `style`, `chore`, or `other`
`confidence`	The model's self-reported confidence level
`reasoning_summary`	A short explanation of why the message fits the diff

This structure is useful for development and analysis. The generated message is the main user-facing output, while the commit type, confidence, and reasoning summary help us debug failure cases and understand how the model interpreted the code change.

5 Technical Details

SmartCommit is implemented as a python notebook prototype. This choice keeps the software easy to run, inspect, and modify without requiring complex local setup. The notebook format is also useful for this project because it allows the user to run each stage of the pipeline sequentially, inspect intermediate outputs, and compare retrieval/generation behavior interactively.

5.1 Development Environment

The prototype is written in Python and executed in a notebook format. The main dependencies are:

Table 4: Libraries and tools used

Library / Tool	Purpose
`datasets`	Load the CommitPackFT dataset from HuggingFace
`pandas`	Store, clean, and inspect processed examples
`difflib`	Generate unified diffs from old and new code
`scikit-learn`	Build TF-IDF vectors and compute cosine similarity
`sentence-transformers`	Encode code diffs into dense embeddings
`faiss-cpu`	Build an efficient nearest-neighbor index for embedding retrieval
`groq`	Call the LLM API for commit message generation
`Google-Colab-Secrets`	Store the Groq API key securely

5.2 Technical Implementation Details

Rather than a linear script, the notebook is structured around three core technical pillars: Scalable data engineering, a dual-path retrieval engine, and a structured generative backend.

5.2.1 Data Engineering and Preprocessing: To manage the computational overhead of the CommitPackFT dataset, the implementation utilizes the Hugging Face datasets library for stream-loading and Pandas for vectorized cleaning.

- Context Truncation: To adhere to the token limits of the MiniLM encoder and the Llama 3 context window, a custom unified diff generator was developed. This function enforces a strict 1,000-character limit per diff, ensuring that the most relevant "hunks" of code are preserved while noisy, repetitive changes are discarded.
- Attribute Standardization: The preprocessing logic transforms disparate dataset fields into a unified schema, specifically generating standardized diff strings from raw file contents using the difflib logic, which is essential for consistent vectorization later in the pipeline.

5.2.2 Retrieval Optimization and Tuning: While the architecture of the dual-path system is defined in Section 3, the implementation required specific hyperparameter tuning to balance retrieval precision with computational efficiency:

- Lexical Tuning (n-gram Rationale): During the implementation of the TfidfVectorizer, we transitioned from a standard unigram model to a (1, 2) n-gram range. This decision was driven by the need to capture "syntactic signatures"—sequences like def __init__ or try: except:. In codebase retrieval, a single token (like def) is too frequent to be discriminative; however, the bigram def test_ provides a strong signal that the current diff belongs to a unit-testing context, allowing the TF-IDF path to prioritize structurally similar commits.
- Semantic Latency Management: For the semantic path, we evaluated the trade-off between embedding dimensions and search latency. By selecting the 384-dimensional all-MiniLM-L6-v2 model, we achieved a "sweet spot": the vectors are dense enough to capture the functional intent of a diff (e.g., "refactoring a loop" vs. "updating a dependency") but small enough to reside in a flat FAISS index within Colab's standard system RAM. This ensures that the top_k nearest neighbors are identified in under 100ms, a critical requirement for a tool intended for real-time developer use.
- Index Isolation: A key implementation detail in the retrieval logic is the handling of the exclude_idx parameter. During testing, the system programmatically masks the query's own index within the FAISS and TF-IDF matrices. This ensures that the evaluation reflects a true "unseen" retrieval scenario, preventing the LLM from simply receiving the ground-truth message as a few-shot example.

5.2.3 Generative Orchestration and Output Safety: The final stage of the pipeline integrates with the Groq LPU to achieve sub-second inference speeds.

- JSON Schema Enforcement: Unlike standard chatbots, our implementation uses a structured output pattern. The prompt explicitly instructs the model to return a JSON object containing the commit message, commit type, confidence, and reasoning summary. This ensures the output can be programmatically parsed and integrated into developer tools without the risk of "chatty" or irrelevant LLM conversational filler.
- Hold-out Logic: To maintain the integrity of our evaluation, the retrieval functions include an exclude_idx parameter. This prevents the "leakage" of ground-truth data during testing, forcing the model to rely on truly similar historical context rather than simply retrieving the exact answer it was provided.

5.3 Key Functions

The core implementation consists of several main functions.

Table 5: Main system functions

Function	Purpose
`make_unified_diff(...)`	Converts old/new code contents into a unified diff string
`retrieve_similar_diffs(...)`	Retrieves top-k examples using TF-IDF and cosine similarity
`retrieve_similar_diffs_embedding(...)`	Retrieves top-k examples using SentenceTransformer embeddings and FAISS
`build_commit_prompt(...)`	Creates the prompt using retrieved examples and the input diff
`generate_smart_commit(...)`	Runs retrieval, calls the LLM, parses the response, and returns diagnostic output

5.4 Software Artifact

The main deliverable for the Development Track is the working software prototype. In our case, the software artifact is the notebook containing the complete pipeline from dataset loading to generated commit message output. The notebook can be used both as a demo and as an experimental environment for comparing retrieval methods.

6 Usage Instructions

This section explains how to run and use the SmartCommit prototype. The current system is implemented as a notebook, so users

can run the pipeline through notebook cells without setting up a full local environment.

6.1 Accessing the Code

The source code for the project is available at: https://github.com/gautam8-uofi/Smart-Git-Commit-Message-Pipeline

6.2 Setup

To use SmartCommit, open the Colab notebook and run the setup cells. The notebook installs the required Python libraries and imports the necessary packages. The user also needs a Groq API key for the LLM generation step. In Google Colab, the key can be stored using Colab Secrets under the name:

```
GROQ_API_KEY
```

This allows the notebook to access the API key securely without hardcoding it into the source code.

6.3 Running the Pipeline

After setup, the user runs the notebook cells in order. This allows the user to verify each step before running the full end-to-end system.

6.4 Generating a Commit Message

To generate a commit message, the user provides an input diff to the generation function. In the notebook, this can be done by sampling a random example from the held-out evaluation set:

```
import random

test_idx = random.randint(0, len(eval_df) - 1)

current_diff = eval_df.loc[test_idx, "diff"]
reference_message = eval_df.loc[test_idx, "commit_message"]

result = generate_smart_commit(
    current_diff=current_diff,
   retriever_function=retrieve_similar_diffs_embedding,
    top_k=3
)
```

The user can also switch retrieval methods by changing the retriever function:

```
retriever_function=retrieve_similar_diffs
```

for TF-IDF retrieval, or:

```
retriever_function=retrieve_similar_diffs_embedding
```

for embedding-based retrieval.

6.5 Interpreting the Output

The output includes both the generated commit message and diagnostic information. A typical output display includes:

```
Input diff:
[original code diff]

Reference message:
[human-written commit message from held-out data]

Generated message:
[LLM-generated commit message]

Commit type:
[predicted type, such as fix/refactor/test/docs]

Confidence:
[high/medium/low]

Reasoning:
[short explanation from the model]

Retrieved examples:
[top-k similar historical diffs and commit messages]
```

The generated commit message is the main result intended for the user. The reference message is shown only when using a held-out evaluation example from the dataset. The retrieved examples and reasoning summary are included to help users and developers inspect why the model may have produced a particular message.

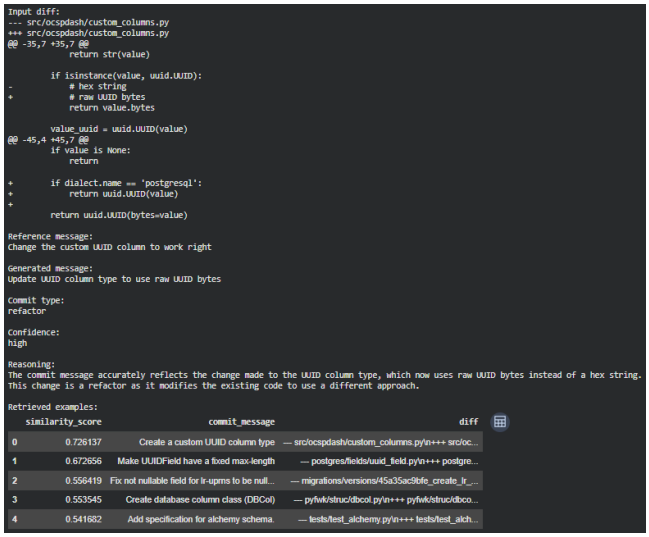

Figure 2: Example of a pipeline final output

7 Evaluation

We evaluate SmartCommit from both the information retrieval perspective and the software usefulness perspective. Since the system depends on retrieving useful examples before generating a commit message, evaluation should consider both the quality of retrieval and the quality of the final generated output.

7.1 Evaluation Goals

The evaluation is designed to answer the following questions:

(1) **Retrieval quality:** Are the retrieved diffs meaningfully similar to the input diff?
(2) **Generation quality:** Does the generated commit message accurately describe the input diff?
(3) **Usefulness:** Would the generated message be useful to a developer writing a commit?

(4) **Method comparison:** Does embedding-based retrieval produce better context than the TF-IDF baseline?

These questions reflect the two main components of the system: the retrieval module and the LLM generation module.

7.2 Retrieval Evaluation

We performed a lightweight evaluation of the retrieval component to compare the behavior of the TF-IDF baseline and the embedding-based FAISS retriever. Since the retrieval dataset does not provide explicit ground-truth relevance labels for which historical diffs should be retrieved for each query, we evaluate retrieval using a combination of similarity score analysis and manual inspection.

For this evaluation, we randomly sampled held-out diffs from the evaluation split. These diffs were not included in the retrieval index, so they represent unseen queries. For each query, we retrieved the top-3 most similar examples using both TF-IDF retrieval and embedding-based retrieval. We then recorded the top-1 similarity score and the average similarity score of the top-3 retrieved examples for each method.

The similarity scores provide a rough indication of how close the retrieved examples are to the query under each retrieval model. However, the scores should not be interpreted as directly comparable across TF-IDF and embedding retrieval, because the two methods use different representations and may produce different score distributions. Instead, we use the scores to summarize retrieval behavior within each method and combine them with qualitative manual inspection.

The retrieval score summary is shown in the table below:

Table 6: Average retrieval similarity scores

Retrieval Method	Average Top-1 Similarity	Average Top-3 Similarity
TF-IDF	0.537639	0.454025
Embedding + FAISS	0.739334	0.692941

As shown in the retrieval score summary, the embedding-based FAISS retriever produced higher average similarity scores than the TF-IDF baseline, with an average top-1 similarity of 0.739 compared to 0.538 for TF-IDF, and an average top-3 similarity of 0.693 compared to 0.454. This suggests that the embedding retriever is finding examples that are closer to the query diff within its representation space. However, because TF-IDF and embedding retrieval use different vector representations, the raw similarity scores are not directly comparable as absolute measures of relevance. Therefore, we interpret these results as descriptive evidence of retrieval behavior, and combine them with qualitative inspection of retrieved examples.

In addition to the similarity score summary, we manually inspected several retrieval examples to better understand the behavior of the two methods. This qualitative inspection is important because raw similarity scores alone do not fully determine whether a retrieved example is useful for commit message generation. We selected examples that illustrate different retrieval patterns, including cases where embedding retrieval performs better and cases where both methods perform similarly.

A few qualitative examples are shown in the table below:

Table 7: Qualitative comparison of retrieved messages

Field	Content
Query Reference Message	`Add new urls file for votes`

TF-IDF Retrieved Message(s)	`Fix a couple of redirect URLs;`
	`Remove special topics and locations from URLs;`
	`Change bill detail page to use session and identifier`

Embedding Retrieved Message(s)	`Add URL scheme for votes app;`
	`Fix vote app URL patterns;`
	`Add votes URL scheme to main URL scheme`

Observation	Embedding retrieval gives a more specific match to the query intent. TF-IDF retrieves URL-related examples, but they are more general, while the embedding method retrieves examples directly related to vote URL patterns and schemes.
Query Reference Message	`Create a new tour example`

TF-IDF Retrieved Message(s)	`Add an example tour for DriverJS;`
	`Add Bootstrap Google Tour example;`
	`Update a SeleniumBase tour example`

Embedding Retrieved Message(s)	`Add an example tour for DriverJS;`
	`Add Bootstrap Google Tour example;`
	`Add an example test to run`

Observation	Both methods perform well because the key concept, "tour example," appears clearly in the query and in the retrieval set. This shows that TF-IDF can be highly effective when there is strong lexical overlap, while embedding retrieval returns a similarly relevant set.
Query Reference Message	`Add missing migration for d4db9f29f`

TF-IDF Retrieved Message(s)	`Increase length of society ID;`
	`Create appropriate migrations for changes in models;`
	`Increase limit on data log model for larger urls`

Embedding Retrieved Message(s)	`Add a migration for common;`
	`Create appropriate migrations for changes in models;`
	`Make migration for item visibility change`

Observation	Embedding retrieval provides a stronger set of migration-related examples overall. TF-IDF retrieves some database/model-related changes, but two of the retrieved messages focus on field length or limits rather than migrations specifically. Embedding retrieval better captures the high-level change type of adding or creating migrations.

These examples suggest that the embedding-based retriever often provides more semantically focused examples, especially when the query is about a higher-level change type such as adding URL routes or creating migrations. However, the second example also shows that TF-IDF remains a strong and interpretable baseline when the query contains distinctive terms that appear in similar historical commits. Overall, the qualitative results support the use of embedding-based retrieval as the stronger retrieval method for the final RAG pipeline, while TF-IDF remains useful as a simple baseline for comparison.

7.3 Generation Evaluation

For generation evaluation, we compare the generated commit message against the human reference message from the held-out evaluation set. Since there can be multiple valid commit messages for the same diff, exact string matching is too strict. A generated message may be useful even if it does not use the same wording as the reference.

We therefore evaluate generation quality using a combination of qualitative examples, semantic similarity and human judgement.

An example of a generated commit message that we deemed was good:

```
Input diff:
--- src/ocspdash/custom_columns.py
```

```
+++ src/ocspdash/custom_columns.py
@@ -35,7 +35,7 @@
             return str(value)

         if isinstance(value, uuid.UUID):
-            # hex string
+            # raw UUID bytes
             return value.bytes

         value_uuid = uuid.UUID(value)
@@ -45,4 +45,7 @@
         if value is None:
             return

+        if dialect.name == 'postgresql':
+            return uuid.UUID(value)
+
         return uuid.UUID(bytes=value)

Reference message:
Change the custom UUID column to work right

Generated message:
Update UUID serialization to raw bytes
```

7.4 Evaluation Discussion

Preliminary examples suggest that SmartCommit can generate concise and meaningful commit messages from held-out diffs. The retrieved examples often provide useful style and context for the LLM. TF-IDF retrieval tends to retrieve examples with stronger lexical overlap, while embedding-based retrieval may retrieve examples with more similar high-level intent. However, generation quality still depends on the relevance of the retrieved examples and the clarity of the input diff. Some generated messages may be overly broad or may include details that are not fully supported by the diff.

8 Limitations and Future Work

Although SmartCommit demonstrates a working retrieval-augmented pipeline for commit message generation, the current prototype has several limitations. These limitations mainly come from the difficulty of interpreting code diffs, the quality of retrieved examples, and the early-stage nature of the software interface.

8.1 Limitations

One major limitation is that raw code diffs can be noisy or ambiguous. A diff may contain formatting changes, renamed variables, refactoring edits, or multiple unrelated changes in the same commit. Even after preprocessing, it can be difficult for the system to infer the true intent behind the change. This may lead to generated commit messages that are technically related to the diff but too broad or incomplete.

A second limitation is that retrieval quality strongly affects generation quality. If the retrieved examples are not actually relevant to the input diff, the LLM may be guided by poor context. TF-IDF retrieval may overemphasize exact token overlap, while embedding-based retrieval may retrieve semantically broad examples that are not specific enough. This means that the retrieval component remains a key bottleneck in the system.

A third limitation is that LLM-generated commit messages may occasionally include unsupported details. Even with a constrained prompt, the model may infer intent that is not clearly present in the diff. For this reason, SmartCommit should be treated as an assistive tool rather than an automatic replacement for human-written commit messages.

Another limitation is that the current implementation focuses only on the Python split of the dataset. This makes the system more consistent for experimentation, but it does not fully test whether the approach generalizes to other programming languages. Different languages may have different syntax, conventions, and commit message patterns.

Finally, the current prototype is implemented as a notebook. This is useful for experimentation and demonstration, but it is less convenient than a fully integrated developer tool. A production-ready version would ideally be available as a command-line tool, Git hook, IDE extension, or web application.

8.2 Future Work

There are several directions for improving SmartCommit beyond the current prototype.

First, we could improve diff preprocessing and representation. Future versions could better detect and remove formatting-only changes, separate unrelated changes within a large diff, and identify important code elements such as modified functions, class names, imported libraries, or changed error-handling logic. This would give both the retriever and the LLM a cleaner and more informative representation of the input diff.

Second, we could improve retrieval quality. One possible extension is to add a reranking step after the initial top-k retrieval. For example, the system could first retrieve a larger candidate set using TF-IDF or FAISS, then use a stronger model to rerank the candidates based on relevance to the input diff. This may improve the quality of examples passed to the LLM and reduce cases where retrieved examples are only superficially similar.

Third, we could incorporate richer project context beyond the code diff itself. In the current system, the input mainly consists of the changed code and retrieved historical examples. However, in real software development, commits are often connected to user stories, issue tickets, bug reports, feature requests, or pull request descriptions. If we can find or construct datasets that link commits to user stories or issue descriptions, the system could use this additional context to generate commit messages that better capture the motivation behind the change, not just the surface-level code edits.

Fourth, we could expand the evaluation through a more systematic LLM-as-a-judge framework. Commit messages are difficult to evaluate because multiple different messages can be valid for the same diff, and exact lexical overlap with a reference message may unfairly penalize good alternatives. A judge model could be given the input diff, the human reference message, and the generated message, then rate the output on criteria such as accuracy, clarity,

usefulness, conciseness, and whether the message is supported by the diff. This would provide a more flexible evaluation method to complement similarity metrics and manual inspection.

Fifth, we could conduct a larger human evaluation with developers. Human participants could compare generated messages from different methods, such as LLM-only generation, TF-IDF RAG, and embedding-based RAG. They could rate each output based on whether it would be useful in an actual commit workflow. This would help us better understand the practical value of SmartCommit from the user's perspective.

Sixth, we could extend the system beyond Python. Supporting multiple programming languages would make the tool more practical and would allow us to study whether retrieval-augmented generation performs consistently across different language ecosystems. Different programming languages may require different preprocessing strategies and may also have different commit message conventions.

Finally, we could improve the software interface. The current Colab notebook is suitable as a prototype and demonstration environment, but future versions could be packaged as a command-line tool that reads the currently staged Git diff, generates commit message suggestions, and allows the user to copy or edit the result. A more advanced version could be implemented as a Git hook, VS Code extension, GitHub integration, or lightweight web application.

9 Team Contributions

The project was completed by a two-person team. Both team members contributed to the design, implementation, evaluation planning, and documentation of SmartCommit.

Jonathon Eng Kiat Low contributed to the retrieval and indexing components of the system. He worked on implementing and analyzing the TF-IDF baseline and embedding-based retrieval approach, including the use of SentenceTransformers and FAISS for nearest-neighbor search. He also contributed to diff preprocessing decisions, evaluation design, and the core technical narrative of the report.

Aryaman Gautam contributed to dataset collection, dataset cleaning, tooling integration, and demo preparation. He worked on loading the CommitPackFT Python split, generating unified diffs from old and new code contents, cleaning and preparing the processed dataset, creating the held-out evaluation split, and integrating the LLM generation function into the end-to-end Colab pipeline. He also contributed to the setup of output displays for analysis, including input diffs, retrieved examples, reference messages, and generated commit messages.